Report

Лабораторная работа 4

1 Структуры данных

Стек реализован на основе двусвязного кольцевого списка ядра (`struct list_head`). Каждый элемент стека определен структурой `stack_node` :

```
struct stack_node {
    struct list_head list;
    int value;
};

struct int_stack {
    struct list_head head;
    int size;
    int max_size;
    struct mutex lock;
};
```

```
Каждый файловый дескриптор имеет отдельный стек. При открытии устройства в `open` под
него выделяется память, структура инициализируется и сохраняется в `filp-
>private_data`. При закрытии `release` освобождает все узлы списка и саму структуру.
```

При записи значение сначала копируется из пространства пользователя через `copy_from_user` , и только потом захватывается мьютекс, потому что функция копирования может заблокироваться. При чтении из пустого стека возвращается 0 байт и интерпретируется пользовательской утилитой как NULL.

Управление размером стека выполняется через `ioctl` : команда `SET_SIZE` устанавливает максимальное число элементов (при условии, что новый лимит не меньше текущего количества), `GET_SIZE` возвращает текущий размер. Все операции, изменяющие состояние стека, защищены мьютексом для корректной работы при конкурентном доступе.

2. Синхронизация

Для защиты стека используется мьютекс. Мьютекс выбран вместо спинлока, поскольку критические секции содержат `kmalloc` , он может заблокироваться, а спинлоки удерживать при блокировке запрещено.

Схема блокировок:

Операция	Захват мьютекса	Освобождение
`read` (pop)	перед проверкой списка	после удаления узла
`write` (push)	перед проверкой размера	после вставки узла
`ioctl SET_SIZE`	перед изменением `max_size`	после записи
`ioctl GET_SIZE`	перед чтением `size`	после чтения

`copy_to_user` / `copy_from_user` выполняются вне мьютекса для минимизации времени удержания блокировки.

3. Коды ошибок

Ситуация	Код	Значение
Pop из пустого стека	`return 0`	`read()` вернул 0 байт, интерпретируется как NULL
Push в полный стек	`-ERANGE`	Стек заполнен
`SET_SIZE` ≤ 0	`-EINVAL`	Некорректный аргумент
`SET_SIZE` < текущий размер	`-ERANGE`	Нельзя установить лимит меньше текущей заполненности
Неизвестная команда `ioctl`	`-ENOTTY`	Стандартный код для неподдерживаемой команды `ioctl`
Недостаточно памяти	`-ENOMEM`	`kmalloc` вернул `NULL`
Ошибка копирования user - kernel	`-EFAULT`	Неверный указатель пользовательского пространства

4. Пользовательская утилита

Утилита состоит из двух файлов:

- `kernel_stack.py` — CLI, разбирает аргументы и отправляет команды демону через Unix-сокет `/tmp/kernel_stack.sock`
- `daemon.py` — фоновый процесс, держит `fd` открытым и выполняет системные вызовы

Почему демон? Каждый `open("/dev/int_stack")` создаёт новый пустой стек. Чтобы состояние сохранялось между вызовами `kernel_stack.py push 1` и `kernel_stack.py push 2`, нужно держать один `fd` открытым, этим занимается демон.

Демон запускается автоматически при первом обращении и живёт до явного `kernel_stack.py stop`.

5. Демонстрация работы

Загрузка модуля

```
anis@deb:~/int_stack/lab4/kernel$ make
make -C /lib/modules/6.12.74+deb13+1-amd64/build M=/home/anis/int_stack/lab4/kernel modules
make[1]: Entering directory '/usr/src/linux-headers-6.12.74+deb13+1-amd64'
  CC [M]  /home/anis/int_stack/lab4/kernel/int_stack.o
  MODPOST /home/anis/int_stack/lab4/kernel/Module.symvers
  CC [M]  /home/anis/int_stack/lab4/kernel/int_stack.mod.o
  CC [M]  /home/anis/int_stack/lab4/kernel/.module-common.o
  LD [M]  /home/anis/int_stack/lab4/kernel/int_stack.ko
  BTF [M] /home/anis/int_stack/lab4/kernel/int_stack.ko
make[1]: Leaving directory '/usr/src/linux-headers-6.12.74+deb13+1-amd64'
anis@deb:~/int_stack/lab4/kernel$ sudo insmod int_stack.ko
anis@deb:~/int_stack/lab4/kernel$ lsmod | grep int_stack
int_stack              12288  0
anis@deb:~/int_stack/lab4/kernel$ ls -l /dev/int_stack
crw------- 1 root root 244, 0 Apr 11 00:28 /dev/int_stack
```

Базовые операции

```
anis@deb:~/int_stack/lab4/userspace$ sudo python3 kernel_stack.py set-size 2
anis@deb:~/int_stack/lab4/userspace$ sudo python3 kernel_stack.py push 1
anis@deb:~/int_stack/lab4/userspace$ sudo python3 kernel_stack.py push 2
anis@deb:~/int_stack/lab4/userspace$ sudo python3 kernel_stack.py push 3
ERROR: stack is full
anis@deb:~/int_stack/lab4/userspace$ sudo python3 kernel_stack.py pop
2
anis@deb:~/int_stack/lab4/userspace$ sudo python3 kernel_stack.py pop
1
anis@deb:~/int_stack/lab4/userspace$ sudo python3 kernel_stack.py pop
NULL
anis@deb:~/int_stack/lab4/userspace$ sudo python3 kernel_stack.py set-size 3
anis@deb:~/int_stack/lab4/userspace$ sudo python3 kernel_stack.py push 1
anis@deb:~/int_stack/lab4/userspace$ sudo python3 kernel_stack.py push 2
anis@deb:~/int_stack/lab4/userspace$ sudo python3 kernel_stack.py push 3
anis@deb:~/int_stack/lab4/userspace$ sudo python3 kernel_stack.py unwind
3
2
1
anis@deb:~/int_stack/lab4/userspace$ sudo python3 kernel_stack.py set-size 0
ERROR: size should be > 0
anis@deb:~/int_stack/lab4/userspace$ sudo python3 kernel_stack.py set-size -1
ERROR: size should be > 0
anis@deb:~/int_stack/lab4/userspace$ sudo python3 kernel_stack.py set-size 2
anis@deb:~/int_stack/lab4/userspace$ sudo python3 kernel_stack.py push 1
anis@deb:~/int_stack/lab4/userspace$ sudo python3 kernel_stack.py push 2
anis@deb:~/int_stack/lab4/userspace$ sudo python3 kernel_stack.py push 3
ERROR: stack is full
anis@deb:~/int_stack/lab4/userspace$ echo $?
222
anis@deb:~/int_stack/lab4/userspace$
```

Выгрузка модуля

```
anis@deb:~/int_stack/lab4/userspace$ sudo python3 kernel_stack.py stop
anis@deb:~/int_stack/lab4/userspace$ sudo rmmod int_stack
anis@deb:~/int_stack/lab4/userspace$ sudo dmesg | tail -5
[  968.439347] int_stack: opened
[ 2421.164346] int_stack: unloaded
[ 2431.237635] int_stack: loaded
[ 2566.927504] int_stack: opened
[ 2980.427241] int_stack: unloaded
```

Лабораторная работа 5

1. Архитектура

Модуль расширяет лабораторную работу 4: символьное устройство `/dev/int_stack_wk` появляется в системе только при наличии конкретного USB-устройства и исчезает при его отключении. Содержимое стека при этом сохраняется, уже открытые файловые дескрипторы продолжают работать.

В качестве ключа используется USB-адаптер TP-Link:

Параметр	Значение
VID	`0x2357`
PID	`0x011f`

Устройство выбрано намеренно: это сетевой адаптер, а не стандартная периферия вроде мыши или флешки, поэтому ядро не подхватывает его стандартными драйверами раньше нашего модуля. Идентификаторы задаются в таблице `usb_device_id` и регистрируются через `MODULE_DEVICE_TABLE`, это позволяет ядру автоматически вызывать `probe` / `disconnect` при подключении и отключении устройства.

2. USB-драйвер

В модуле добавлен атомарный флаг `atomic_t usb_key_present` и два колбэка:
`key_probe` вызывается при подключении ключа: создаёт `/dev/int_stack_wk` через `device_create` и устанавливает флаг.

`key_disconnect` вызывается при отключении: удаляет устройство через `device_destroy` и сбрасывает флаг. Стек в памяти не трогается -- уже открытые fd продолжают работать.

Все файловые операции в начале проверяют флаг и возвращают `-ENODEV`, если ключ не вставлен:

```
if (!atomic_read(&usb_key_present))
    return -ENODEV;
```

Для флага использован `atomic_t`, операция чтения/записи одного значения не требует мьютекса, `atomic_read` / `atomic_set` гарантируют атомарность без накладных расходов на блокировку.

3. Жизненный цикл устройства

После `insmod cdev` и класс зарегистрированы, но `/dev/int_stack_wk` ещё не существует. Устройство появляется только после подключения ключа и исчезает при его извлечении. Уже открытый fd при этом возвращает `-ENODEV` на любую операцию, но данные стека остаются в памяти, после повторного подключения ключа и нового `open` они доступны.

Демо

Загрузка модуля и подключение ключа

После `insmod` устройство в `/dev` отсутствует. При подключении USB-ключа `key_probe` создаёт `/dev/int_stack_wk`, это видно в выводе `dmesg` и подтверждается повторным `ls /dev | grep int`.

```
anis@deb:~/int_stack/lab5/kernel$ sudo rmmod int_stack_wk
rmmod: ERROR: Module int_stack_wk is not currently loaded
anis@deb:~/int_stack/lab5/kernel$ sudo insmod int_stack_wk.ko
anis@deb:~/int_stack/lab5/kernel$ ls /dev | grep int
anis@deb:~/int_stack/lab5/kernel$ sudo dmesg | tail -5
[ 3446.221406] usb 1-1: New USB device strings: Mfr=1, Product=2, SerialNumber=3
[ 3446.221409] usb 1-1: Product: 802.11ac WLAN Adapter
[ 3446.221412] usb 1-1: Manufacturer: Realtek
[ 3446.221414] usb 1-1: SerialNumber: 00e04c000001
[ 3446.230542] int_stack_wk: USB key inserted, /dev/int_stack_wk available
anis@deb:~/int_stack/lab5/kernel$ ls /dev | grep int
int_stack_wk
```

Работа при подключённом ключе и извлечение

`test.py` выполняет push/pop в цикле. При извлечении USB во время работы операция завершается с `[Errno 19] No such device` (`-ENODEV`). Повторный запуск без ключа падает с `FileNotFoundError` -- устройства в `/dev` нет, `open` невозможен.

```
anis@deb:~/int_stack/lab5/kernel$ sudo python3 ../userspace/test.py
read: 42
read: 42
read: 42
read: 42
read: 42
read: 42
Error: [Errno 19] No such device
anis@deb:~/int_stack/lab5/kernel$ sudo dmesg | tail -5
[ 3446.221414] usb 1-1: SerialNumber: 00e04c000001
[ 3446.230542] int_stack_wk: USB key inserted, /dev/int_stack_wk available
[ 3511.443794] int_stack_wk: opened
[ 3517.408076] usb 1-1: USB disconnect, device number 2
[ 3517.408673] int_stack_wk: USB key removed, /dev/int_stack_wk unavailable
anis@deb:~/int_stack/lab5/kernel$ sudo python3 ../userspace/test.py
Traceback (most recent call last):
  File "/home/anis/int_stack/lab5/kernel/../userspace/test.py", line 5, in <module>
    fd = os.open("/dev/int_stack_wk", os.O_RDWR)
FileNotFoundError: [Errno 2] No such file or directory: '/dev/int_stack_wk'
```

Сохранение стека при извлечении ключа

`test2.py` держит `fd` открытым: пушит 1, 2, 3, затем ключ извлекается, операция возвращает ошибку. После повторной вставки pop возвращает 3, 2, 1 в правильном порядке, что подтверждает сохранность данных.

```
anis@deb:~/int_stack/lab5/kernel$ sudo python3 ../userspace/test2.py
pushed: 1
pushed: 2
pushed: 3
Извлеките USB и нажмите Enter...
Error: USB key is not inserted ([Errno 19] No such device)
Вставьте USB обратно и нажмите Enter...
popped: 3
popped: 2
popped: 1
```